

Ingestion patterns and where data lands

Batch, streaming and incremental loading, the medallion layers, file formats, and a map of the tools that follow.

Almost every ingestion question reduces to one thing: **how does the system know what it has already processed?**

The three ingestion patterns

PATTERN	WHAT IT DOES	TYPICAL LATENCY
Batch	Reads a defined set of data, all at once	Hours or a day
Incremental	Reads only what is new since last time	Minutes to hours
Streaming	Processes records continuously as they arrive	Seconds

ON THE EXAM

The exam names all three. In practice most Databricks ingestion is incremental, and the tooling is built around it.

Where batch and streaming actually differ

BATCH

You define the data

- The job knows the full set of input before it starts
- Runs, finishes, exits
- Re-running reprocesses the same data again
- No memory between runs unless you build one

STREAMING

The data is unbounded

- New data may arrive at any time; there is no 'all of it'
- A checkpoint records progress so the next run resumes
- Re-running continues rather than repeats
- Can be triggered to stop once caught up

THE TIEBREAKER

The real difference is not speed. It is whether progress is remembered. Streaming keeps a checkpoint; batch does not.

What incremental actually requires

- ◆ **A record of what has been processed.** A checkpoint, or a list of loaded files, kept somewhere durable.
- ◆ **Idempotency.** Running the same load twice must not double the data.
- ◆ **A way to discover new input** without listing everything every time, once volumes get large.
- ◆ **Not a filter on a date column.** That is a manual workaround, and it breaks on late-arriving data.

ON THE EXAM

If an option achieves incremental behaviour by filtering on yesterday's date, look for the option that tracks state instead.

The shape of a Databricks pipeline

01

Source

Files, databases, SaaS applications, message buses.

02

Bronze

Raw, as it arrived, appended. Nothing thrown away.

03

Silver

Cleaned, typed, deduplicated, joined. Trustworthy.

04

Gold

Aggregated and shaped for a specific consumer.

LEARN THE LAYER NAMES

This is the medallion architecture. It is the vocabulary of domain three, and the exam uses the layer names without defining them.

What each layer is for

LAYER	CONTAINS	WRITTEN HOW
Bronze	Raw source data, plus ingestion metadata	Append only
Silver	Cleaned, typed, deduplicated, conformed	Merge or overwrite
Gold	Business-level aggregates and metrics	Recomputed or merged

ON THE EXAM

Bronze is append-only on purpose. It is your ability to rebuild everything downstream when a transformation turns out to be wrong.

Three rules that follow from the layers

- ◆ **Do not clean on the way into bronze.** Cleaning belongs in the bronze-to-silver step, where it can be re-run.
- ◆ **Silver is the layer most consumers should use.** It is trustworthy and still detailed.
- ◆ **Gold is allowed to be duplicated.** Several gold tables from one silver table is normal, not a design flaw.
- ◆ **Layers are a convention, not a feature.** There is no CREATE BRONZE TABLE. They are ordinary tables you have named.

ON THE EXAM

A question describing 'raw data with quality problems' is describing bronze, and the fix belongs downstream of it.

File formats you will meet

FORMAT	SHAPE	WHERE IT TURNS UP
CSV	Row-based text, no types	Exports from older systems, manual uploads
JSON	Text, nested, self-describing	APIs, event streams, application logs
Parquet	Columnar, compressed, typed	Analytics interchange, the basis of Delta
Avro	Row-based binary, schema attached	Message buses, Kafka payloads
ORC	Columnar, typed	Legacy Hadoop estates
Delta	Parquet plus a transaction log	Every table you create on Databricks

ON THE EXAM

Columnar formats let a query read only the columns it needs. Row-based formats make it read whole records.

Reading files

- ◆ `spark.read.format("csv").option(...).load(path)` is the general shape for a batch read.
- ◆ **Options carry the details:** header, delimiter, inferSchema, multiLine for JSON.
- ◆ **inferSchema costs a pass over the data** to work out types, which is slow on large inputs.
- ◆ **Supplying a schema is faster and safer.** It also stops types changing silently between runs.

ON THE EXAM

A read with no schema and no inference gives every column as STRING. That is the cause behind a whole family of questions.

Local files and small uploads

- ◆ **Unity Catalog volumes** are the governed home for file data. Grants apply, and access appears in audit logs.
- ◆ **The upload UI** can create a table directly from a small CSV or JSON file.
- ◆ **Workspace files** sit beside your notebooks and suit small reference files and code, not data at scale.
- ◆ **Not the DBFS root.** Deprecated, and readable by everyone in the workspace.

ON THE EXAM

The exam guide explicitly lists local files as an ingestion source. The governed answer is a volume.

Land in storage first, or write straight to a table?

LAND IN CLOUD STORAGE

A raw landing zone

- Original files are kept exactly as delivered
- Ingestion can be re-run after a bug is found
- Needed when the source pushes files to you
- Adds a step and a place for files to pile up

WRITE DIRECTLY TO A TABLE

Straight into Unity Catalog

- Fewer moving parts
- Governed and queryable immediately
- Nothing to re-read if the load logic was wrong
- Suits API and database pulls where the source can be re-queried

THE TIEBREAKER

The question is whether you can go back to the source. If you cannot, keep the files.

The ingestion tools, and which module covers each

TOOL	BEST FOR	MODULE
COPY INTO	Files in cloud storage, modest volumes, SQL-first teams	04
Auto Loader	Files in cloud storage at scale, schema drift, continuous arrival	05
Lakeflow Connect managed	SaaS and database sources, fully handled for you	06
Lakeflow Connect standard	Sources needing more control than managed allows	06
JDBC, ODBC and REST	Databases and APIs pulled from a notebook	07
Partner connectors	Sources Databricks does not cover natively	08

ON THE EXAM

Module 08 is entirely about choosing between these. It carries eight scenarios rather than four.

Three things that go wrong at ingestion

- ◆ **Duplicates.** A load re-runs, or a file is delivered twice, and there is nothing tracking what was seen.
- ◆ **Schema drift.** The source adds a column, and the pipeline either fails or silently drops it.
- ◆ **Small files.** Frequent tiny writes produce thousands of small files, and reads slow to a crawl.
- ◆ **Each has a named owner:** checkpoints for duplicates, schema evolution for drift, OPTIMIZE and liquid clustering for small files.

ON THE EXAM

Recognising which of the three a scenario describes usually names the answer before you have read the options.

Tiebreakers from this module

WHEN A QUESTION SAYS

Files keep arriving and must not be reprocessed

Re-running the load creates duplicates

Keep the raw data exactly as delivered

A BI team needs one specific aggregate

Governed access to uploaded files

Every column arrived as a string

REACH FOR

Incremental, with tracked state

Nothing is tracking processed files

Bronze, append only

Gold, built for that consumer

A Unity Catalog volume

No schema supplied and no inference

ON THE EXAM

Six lines. Each maps a phrase you will actually see in a question to the concept behind it.

A supplier drops CSV files into a cloud storage folder several times a day. The times are irregular. A nightly job runs `spark.read` on the whole folder and overwrites a Delta table with the result.

The folder now holds four years of files. The job takes over three hours and is getting slower each month. The team wants the table to stay current without the job growing indefinitely.

What is the underlying problem?

- ☐ A The job overwrites rather than appends, so it loses history each night
- ☐ B The job reprocesses every file every night, because nothing records what was already loaded
- ☐ C The cluster is undersized for four years of CSV data
- ☐ D CSV is the wrong format and the files should be converted to Parquet

◆ PAUSE AND DECIDE

What is the underlying problem?

A Overwrite loses history

It does overwrite, but it rebuilds from all four years each night, so nothing is actually lost. A real observation that is not the cause.

C The cluster is undersized

A bigger cluster makes the same wasteful work finish faster, and the runtime still grows every month. It treats the symptom.

B It reprocesses everything, because nothing records what was loaded

Correct. This is a batch job pretending to be incremental. The runtime grows with total history rather than with new data.

D CSV should be Parquet

Parquet would read faster, but the job would still reprocess four years of it nightly. Same growth curve, shallower slope.

THE SIGNAL

'Getting slower each month' is the signature. Runtime scaling with accumulated history rather than with new data means no state is being tracked.

A team ingests order events from an API into a bronze table. During ingestion they drop records with a null customer_id, cast the amount column to decimal, and remove three columns nobody uses.

Six months later they discover the cast was wrong: amounts in one currency were being divided by 100. They need to correct the historical figures.

What is the design error?

- ☐ A They should have used a streaming read rather than a batch read
- ☐ B They transformed data on the way into bronze, so the original values no longer exist
- ☐ C They should have partitioned the bronze table by currency
- ☐ D They should have written to silver rather than bronze

◆ PAUSE AND DECIDE

What is the design error?

- A

 Should have used streaming
Batch against streaming is unrelated to this failure. Either one would have destroyed the original values just the same.
- B

 They transformed on the way into bronze
Correct. Bronze exists to hold the source as delivered. Cleaning during ingestion removed the only copy of the true amounts.
- C

 Should have partitioned by currency
Partitioning affects query performance. It does nothing to preserve values that were altered before they were ever written.
- D

 Should have written to silver
Backwards. The table is named bronze but behaves like silver. Renaming it would not bring back the discarded data.

THE SIGNAL

Three transformations happened at ingestion: a filter, a cast and a projection. Each one discarded something bronze was supposed to keep.

An analyst uploads a 40-row CSV of regional sales targets that she maintains by hand. Three pipelines need to join against it, and the platform team requires that all data access is governed and auditable.

She currently emails the file to an engineer, who puts a copy on the DBFS root each quarter.

What should the team do instead?

- ☐ A Keep it on the DBFS root but restrict access with a cluster policy
- ☐ B Have her upload it to a Unity Catalog volume, or create a table from it via the upload UI
- ☐ C Ingest it with Auto Loader so it is tracked incrementally
- ☐ D Store it as a workspace file next to the pipeline notebooks

◆ PAUSE AND DECIDE

What should the team do instead?

A DBFS root with a cluster policy
Cluster policies constrain compute configuration, not file access. The DBFS root stays readable by every workspace user.

C Auto Loader
Machinery for continuous arrival at scale, aimed at a 40-row file replaced quarterly. It solves a problem that does not exist here.

B A Unity Catalog volume, or a table via the upload UI
Correct. Both are governed, auditable, and let the analyst maintain the file herself without an engineer in the loop.

D A workspace file
Reasonable for small reference data, but it is not a Unity Catalog securable, so it does not satisfy the governance requirement.

THE SIGNAL

Two requirements: governed and auditable, and the analyst should not need an engineer. Only B satisfies both.

A team is migrating from an on-premises warehouse. They must load five years of history once, roughly 2 TB in Parquet files, and then keep a daily delta of about 3 GB flowing afterwards.

An engineer proposes building one pipeline that handles both, using the same incremental tool for the history and the daily load.

How should the team think about this?

- ☐ A The proposal is wrong; historical backfill and ongoing ingestion always need separate tools
- ☐ B The proposal is reasonable; an incremental tool processes the backlog first, then continues with daily files
- ☐ C The proposal is wrong; 2 TB is too large to load into a Delta table
- ☐ D The proposal is reasonable, but only if the history is loaded as a stream with a seconds-level trigger

◆ PAUSE AND DECIDE

How should the team think about this?

- A

 Backfill and ongoing always need separate tools
The word 'always' is the giveaway. An incremental tool pointed at a directory treats existing files as its first batch, then continues from the checkpoint.
- B

 Reasonable; it processes the backlog then continues
Correct. The initial run consumes what is already there, records progress, and subsequent runs pick up only new files.
- C

 2 TB is too large for a Delta table
Invented limit. Delta tables routinely hold far more than this.
- D

 Only as a stream with a seconds-level trigger
A seconds-level trigger on a daily 3 GB delta means a cluster running continuously to do nothing most of the time.

THE SIGNAL

Backfill and steady state are the same problem to an incremental tool. The first run is simply a very large batch.

Every tool in the next five modules is answering the same question in a different way: **what have I already seen, and how do I know?**